# Chess Engine (SDL2)
# Computer Programming I Final Project

**Alexandria University**
Faculty of Engineering
Computer and Systems Engineering Department

**Computer Programming I**
Final Project
Fall 2025

# Chess Engine (SDL2)

Computer Programming I – Final Project

Github Repository:
github.com/Joessameldinq/Chess-Master-CSED28-

## Team Members

**Youssef Essam ElDeen Mahmoud ElSaeed**

ID: 24010854

**Abdelwahhab Khaled Mohamed Khamis**

ID: 24010416

## Supervision

Prof. Dr. Marwan Torki

Eng. Karim Alaa

# Contents

# Chapter 1

# Introduction

The SDL2 Chess Engine is a high-performance graphical board game application implemented in C. It evolves the complexity of traditional chess logic into a modern, interactive experience by bridging a robust backend engine with a hardware-accelerated frontend. The application strictly adheres to FIDE regulations, accurately handling technical maneuvers such as kingside/queenside castling, en passant captures, and pawn promotion.

## 1.1 Project Overview

This project was developed as part of the Computer Programming I course final project. It demonstrates the application of fundamental programming concepts including data structures, algorithm design, file I/O, and graphical user interface development using SDL2 libraries.

## 1.2 Features

The chess engine includes the following comprehensive features:

- Graphical chess board using SDL2
- Piece rendering with SDL2_image
- Text rendering with SDL2_ttf
- Sound effects using SDL2_mixer
- Background music making you enthusiastic
- Legal move validation
- Supporting en passant, castling, pawn promotion and stalemate
- The game supports redo and undo till the first move
- Turn-based gameplay
- Offering eight slots to save games and continue them later
- Highlight valid move squares when piece is dragged
- Highlight king square with red when he is in check position

- Highlight last move squares with yellow
- Show several message boxes during gameplay
- You can choose a theme for board out of 12 themes
- Clean modular C codebase
- Adding an icon for the executable game
- Game supports draw by threefold repetition using Zobrist hashing and transposition tables
- Cross-platform support (Windows & Linux)

# Chapter 2

# Technologies Used

## 2.1 Programming Language

The entire project is implemented in **C**, demonstrating proficiency in low-level programming and memory management.

## 2.2 Libraries

The following SDL2 libraries were utilized:

- **SDL2** - Core library for window management and rendering
- **SDL2_ttf** - TrueType font rendering
- **SDL2_image** - Image loading for PNG, JPG formats
- **SDL2_mixer** - Audio and music playback

## 2.3 Build Tool

**Makefile** - Used for automated compilation and linking across different platforms

## 2.4 Project Structure

```
Chess-Master-CSED28/
|-- src/            # Source files
|-- cfg/            # Binary saved games
|-- include/        # Header files
|-- assets/         # Images, fonts, sounds
|-- thirdparty/     # SDL libraries (Windows only)
|-- Makefile        # Build configuration
|-- README.md       # Project documentation
```

# Chapter 3

# Data Structures

## 3.1 Core Enumerations

### 3.1.1 PieceType Enumeration

```
typedef enum
{
    EMPTY = 0,
    PAWN = 1,
    KNIGHT = 2,
    BISHOP = 3,
    ROOK = 4,
    QUEEN = 5,
    KING = 6
} PieceType;
```

### 3.1.2 Color Enumeration

```
typedef enum
{
    NONE = 0,
    WHITE = 1,
    BLACK = 2
} Color;
```

### 3.1.3 GameStatus Enumeration

```
typedef enum
{
    PLAYING,
    CHECK,
    CHECKMATE,
    STALEMATE,
    DRAW_FIFTY_MOVE,
    DRAW_INSUFFICIENT_MATERIAL,
    DRAW_AGREEMENT,
    DRAW_THREEFOLD_REPTITION
} GameStatus;
```

### 3.1.4 MoveType Enumeration

```
typedef enum
{
    NORMAL_MOVE,
    CAPTURE,
    CASTLE_KINGSIDE,
    CASTLE_QUEENSIDE,
    EN_PASSENT,
    PAWN_PROMOTION,
    CAPTURE_AND_PAWN_PROMOTION
} MoveType;
```

## 3.2 Fundamental Structures

### 3.2.1 Position Structure

```
typedef struct
{
    int x;
    int y;
} Position;
```

### 3.2.2 Piece Structure

```
typedef struct
{
    PieceType type;
    Color color;
    bool hasMoved;
} Piece;
```

### 3.2.3 Move Structure

```
typedef struct
{
    Position initial;
    Position final;
    Piece capturedPiece;
    MoveType moveType;
    PieceType promotionPiece;
} Move;
```

## 3.3 Game State Structures

### 3.3.1 GameFlags Structure

```c
typedef struct
{
    bool pawnPromotionMade;
    bool castlingMade;
    bool enpassentMade;
} GameFlags;
```

### 3.3.2 Game Structure

```c
typedef struct
{
    Piece board[BOARD_SIZE][BOARD_SIZE];
    Piece capturedWhitePieces[16];
    Piece capturedBlackPieces[16];
    int halfMoveClock;
    GameStatus status;
    Color currentPlayer;
    Position enPassentTarget;
    GameFlags currentFlag;
    bool enPassentAvailable;
    uint64_t currentHash;
    uint64_t hashHistory[1024];
    int hashCount;
} Game;
```

## 3.4 Stack Structure for Undo/Redo

### 3.4.1 Node Structure

```c
typedef struct Node
{
    Game curGame;
    struct Node* nextGame;
} Node;
```

## 3.5 Zobrist Hashing Structure

### 3.5.1 ZobristTables Structure

. Using the libraray stdint.h

```c
typedef struct {
    uint64_t pieces[64][12];
    uint64_t castling[4];
    uint64_t enpassant[8];
    uint64_t sideToMove;
} ZobristTables;
```

# Chapter 4

# Moving Logic

## 4.1 Piece Validation Functions

The engine implements individual validation functions for each piece type to ensure moves comply with chess rules.

### 4.1.1 Validation Function Summary

| **Function** | **Functionality** |
|---|---|
| `isValidPawn` | Checks single/double steps forward, diagonal capture, and En Passant logic |
| `isValidRook` | Validates horizontal and vertical movement |
| `isValidBishop` | Validates diagonal movement by checking if absolute row difference equals column difference |
| `isValidKnight` | Checks for the unique L-shape (2×1 or 1×2 squares) |
| `isValidQueen` | Combines logic of both Rook and Bishop |
| `isValidKing` | Handles standard 1-square movement in any direction |
| `isValidCastling` | Specialized check for King/Rook safety during castling |

Table 4.1: Piece Validation Functions

## 4.2 Move Validation Utilities

### 4.2.1 Key Utility Functions

| Function | Functionality |
|---|---|
| `canPieceMoveTo` | Checks if piece can move to position geometrically |
| `isPathClear` | Verifies all squares between initial and final positions are empty |
| `simulateMoveAndShowIfInCheck` | Creates backup and checks if move leaves king in check |
| `isLegalMove` | Validates piece-specific movement rules |
| `isValidMove` | Final validation before applying move |
| `isSquareAttacked` | Returns true if position is attacked by opponent |
| `applyMove` | Applies the validated move to game state |

Table 4.2: Move Validation Utility Functions

## 4.3 Tiered Validation System

The engine employs a three-tier validation approach:

1. **Geometric Validation**: Checks if piece can physically reach target square
2. **Rule Validation**: Ensures move complies with piece-specific chess rules
3. **King Safety Validation**: Verifies move doesn't leave king in check

### 4.3.1 Preventing Infinite Recursion

The validation system prevents circular dependencies through careful function layering:

- `isValidMove` simulates moves and calls `inCheck()`
- `inCheck()` calls `isSquareAttacked()`
- `isSquareAttacked()` calls lightweight `canPieceMoveTo()`
- `canPieceMoveTo()` does not check for checks, terminating the chain

## 4.4 Move Simulation Pattern

The "Simulate-Verify-Restore" pattern ensures king safety:

1. **Simulation**: Game state copied to temporary structure
2. **Verification**: `applyMove()` executed on copy, `inCheck()` queried
3. **Result**: If king under attack, move flagged as illegal before modifying actual board

# Chapter 5

# Ending Conditions

## 5.1 Game State Evaluation Functions

### 5.1.1 Core Functions

| Function | Functionality |
|---|---|
| `inCheck` | Checks if current player's king is under attack |
| `inCheckMate` | Verifies if king is in check with no legal escape moves |
| `isStalemate` | Determines if player has no legal moves but king not in check |
| `fiftyMovesRule` | Checks if 50 moves made without pawn move or capture |
| `isDeadPosition` | Evaluates if insufficient material for either player to win |
| `isThreeFoldRepetition` | Detects if same position repeated three times |

Table 5.1: Game State Evaluation Functions

### 5.1.2 Utility Functions

| Function | Functionality |
|---|---|
| `getSquareColor` | Returns color of square for bishop position analysis |
| `findKingPosition` | Locates king of specified color on board |
| `computeGameStatus` | Updates game state after move application |
| `initZobristTables` | Initializes random number tables for hashing |
| `getPieceIndex` | Returns unique index for piece in Zobrist table |
| `computePositionHash` | Calculates hash value for current position |

Table 5.2: Ending Condition Utility Functions

## 5.2 Dead Position Detection

The `isDeadPosition` function handles material-based draws with high accuracy:

### 5.2.1 Detected Scenarios

- King vs. King
- King & Bishop vs. King

- King & Knight vs. King
- King & Bishops (same color) vs. King & Bishop

### 5.2.2 Limitations

- Does not detect geometric deadlocks (blocked pawn chains)
- Relies on Fifty-Move Rule for complex blocked positions
- King & Knight vs. King & Knight not flagged (checkmate possible with blunder)

## 5.3 Zobrist Hashing for Threefold Repetition

The engine implements Zobrist hashing to efficiently detect position repetitions:

- Each position assigned unique 64-bit hash value
- Hash history maintained in array
- Threefold repetition detected by comparing current hash with history

# Chapter 6

# Save and Load System

## 6.1 Binary Format Storage

The engine saves and loads games in binary format, allowing complete game state preservation.

## 6.2 Architecture Benefits

The Game structure design enables:

- Complete game snapshot in single struct
- Direct binary serialization
- Fast save/load operations
- Multiple save slots support

## 6.3 Static Helper Functions

### 6.3.1 Write Functions

```
static bool writeInt(FILE *fp, int v)
static bool writeBool(FILE *fp, bool v)
static bool writeEnum(FILE *fp, int v)
static bool writeU64(FILE *fp,  uint_64 v)
```

### 6.3.2 Read Functions

```
static bool readInt(FILE *fp, int *v)
static bool readBool(FILE *fp, bool *v)
static bool readEnum(FILE *fp, int *v)
static bool readU64(FILE *fp, uint_64 *v)
```

## 6.4 Core Save/Load Functions

### 6.4.1 Save Game Function

```
bool saveGame(FILE *fp, const Game *g)
```

Features:

- Writes magic header and version number
- Serializes entire Game structure
- Validates each write operation
- Returns success status

### 6.4.2 Load Game Function

```
bool loadGame(FILE *fp, Game *g)
```

Features:

- Validates magic header and version
- Deserializes Game structure in correct order
- Validates each read operation
- Returns success status

## 6.5 Save Slot System

The engine provides eight save slots:

- Stored in `cfg/` directory
- Message box for slot selection
- Extensible for additional slots
- Binary file format for efficiency

# Chapter 7

# Undo and Redo System

## 7.1 Stack-Based Architecture

The undo/redo system uses two stacks to maintain game history:

- **Game Stack**: Current and previous game states
- **Redo Stack**: States that can be reapplied

## 7.2 Stack Operations

### 7.2.1 Core Functions

| Function | Functionality |
|---|---|
| `push` | Adds game state to top of stack |
| `pop` | Removes and returns game state from top of stack |
| `initializeStack` | Creates new empty stack |
| `isEmptyStack` | Checks if stack has any elements |
| `clearStack` | Removes all elements from stack |

Table 7.1: Stack Operations

## 7.3 Undo/Redo Logic

### 7.3.1 Undo Operation

```
1 bool undoMove(Node **gameStack, Node **redoStack)
```

Process:

1. Verify at least two states exist in game stack
2. Pop current state from game stack
3. Push popped state to redo stack
4. Return to previous game state

### 7.3.2 Redo Operation

```
bool redoMove(Node **gameStack, Node **redoStack)
```

Process:

1. Verify redo stack is not empty
2. Pop state from redo stack
3. Push state back to game stack
4. Restore game to that state

## 7.4 Implementation Notes

- Undo/redo history not saved between sessions
- Can undo all the way to first move
- Redo stack cleared when new move made
- Memory managed efficiently through linked list

# Chapter 8

# Graphical User Interface

## 8.1 GUI Architecture

The interface consists of two main components:

1. **Main Menu**: Game selection and options
2. **Game Mode**: Active gameplay interface

## 8.2 Main Menu

### 8.2.1 Features

- New Game button
- Load Game button (shows 8 save slots)
- Quit button
- Button highlighting with ocean blue color on click

## 8.3 Game Mode Interface

### 8.3.1 Control Buttons

| **Button** | **Action** |
|---|---|
| Undo | Reverts to previous turn |
| Redo | Reapplies undone move |
| Save | Opens save slot selection |
| Back | Returns to Main Menu |
| Draw | Offers draw to opponent |
| Arrow Forward/Back | Changes board theme |

Table 8.1: Game Mode Control Buttons

### 8.3.2 Visual Feedback

- **Green highlighting**: Valid move squares when piece dragged

- **Yellow highlighting**: Last move's start and end squares
- **Red highlighting**: King square when in check
- **Captured pieces display**: Two columns showing captured pieces
- **Turn indicator**: Shows current player
- **Half-move clock**: Displays move count for fifty-move rule

## 8.4 Sound Effects

The game provides distinct audio feedback: Background music can be paused

- Normal moves and captures
- Castling
- En passant
- Pawn promotion
- Check
- Checkmate
- Stalemate
- Invalid move beep
- Draw offer
- Background music (can be paused)

## 8.5 GUI Data Structures

### 8.5.1 Button Structure

```
typedef struct
{
    SDL_Texture *texture;
    SDL_Rect rect;
} Button;
```

### 8.5.2 App Structure

```
typedef struct {
    SDL_Window *window;
    SDL_Renderer *renderer;
    bool running;
    GameState currentScreen;
    Game game;
    Node *gamestack;
    Node *redostack;
    int windowWidth;
    int windowHeight;
    int boardOffset;
    int squareSize;
    int buttonWidth;
    int buttonHeight;
    int buttonSpacing;
} App;
```

### 8.5.3 GameGUI Structure

Contains all game mode interface elements including buttons, textures, sounds, and dynamic dimensions for window resizing support.

## 8.6 Event Handling

The GUI uses `SDL_WaitEvent` for efficient CPU usage:

- Blocks until user input received
- Reduces CPU consumption compared to polling
- Appropriate for turn-based game
- Maintains responsive interface

### 8.6.1 SDL_WaitEvent vs SDL_PollEvent

| **Aspect** | **SDL_WaitEvent** | **SDL_PollEvent** |
|---|---|---|
| Behavior | Blocks until event occurs | Returns immediately if no events |
| CPU Usage | Very efficient, yields to OS | Constantly uses CPU cycles |
| Use Case | Event-driven applications | Real-time games requiring continuous updates |
| Battery Impact | Minimal | Higher consumption |

Table 8.2: Event Handling Comparison

# Chapter 9

# Build and Run Instructions

## 9.1 Windows Setup

### 9.1.1 Requirements

- GCC (MinGW)
- SDL2 libraries

### 9.1.2 Required Libraries

| Library | Version | Source |
|---|---|---|
| SDL2 | 2.24.0 | github.com/libsdl-org/SDL/releases |
| SDL2_ttf | 2.24.0 | github.com/libsdl-org/SDL_ttf/releases |
| SDL2_image | 2.7.1 | github.com/libsdl-org/SDL_image/releases |
| SDL2_mixer | 2.8.1 | github.com/libsdl-org/SDL_mixer/releases |

Table 9.1: SDL2 Library Requirements

### 9.1.3 Build Steps

```
git clone https://github.com/Joessameldinq/Chess-Master-CSED28-.git
cd Chess-Master-CSED28-
# Create thirdparty folder and copy SDL libraries
# Copy DLL files to executable directory
make clean
## To Run GUI
make run-GUI

## To Run Console
make run-Console
```

## 9.2 Linux (Ubuntu) Setup

### 9.2.1 Requirements

- GCC: GNU C Compiler
- pkg-config: Helps compiler find SDL2 paths

### 9.2.2 Install Development Tools

```
sudo apt update
sudo apt install build-essential pkg-config
```

### 9.2.3 Install SDL2 Libraries

```
sudo apt install libsdl2-dev libsdl2-image-dev \
                 libsdl2-ttf-dev libsdl2-mixer-dev
```

### 9.2.4 Compile and Run

- For GUI

```
gcc ./src/gui/*.c ./src/common/*.c -o ChessGui `sdl2-config --cflags --
    libs` -lSDL2_image -lSDL2_mixer -lSDL2_ttf
./ChessGui
```

- For Console

```
gcc ./src/console/*.c ./src/common/*.c -o ChessConsole `sdl2-config --
    cflags --libs` -lSDL2_image -lSDL2_mixer -lSDL2_ttf
./ChessConsole
```

# Chapter 10

# User Manual

## 10.1 Getting Started

### 10.1.1 Launching the Game

Upon launching, the Main Menu appears with three options:

1. **New Game**: Starts fresh match
2. **Load Game**: Resumes saved match
3. **Quit**: Exits application

### 10.1.2 Basic Controls

- **Select Piece**: Click and hold left mouse button
- **Move Piece**: Drag to target square
- **Visual Aids**:
  - Valid moves highlighted in green
  - Last move highlighted in yellow
  - King in check highlighted in red

## 10.2 Special Chess Rules

### 10.2.1 Castling

- King and Rook must not have moved
- Path between them must be clear
- King cannot castle through check
- Move king two squares toward rook

### 10.2.2 En Passant

- Special pawn capture
- Only available immediately after opponent moves pawn two squares
- Must be executed on next turn or opportunity lost

### 10.2.3 Pawn Promotion

- Occurs when pawn reaches 8th rank
- Menu appears to choose: Queen, Rook, Bishop, or Knight
- Most commonly promoted to Queen

### 10.2.4 Draw Conditions

The engine automatically detects:

- Stalemate
- Insufficient Material
- Fifty-Move Rule
- Threefold Repetition
- Draw by Agreement

## 10.3 Saving and Loading

### 10.3.1 To Save

1. Click Save button
2. Select slot (1-8) from message box
3. Game state saved to `cfg/` folder

### 10.3.2 To Load

1. Click Load Game from Main Menu
2. Select previously used slot
3. Game restored to exact saved state

**Note**: Undo/Redo history not preserved between saved games.

## 10.4 Troubleshooting

### 10.4.1 Common Issues

- **Window Unresponsive**: Normal behavior - waiting for input
- **Invalid Moves**: Ensure move doesn't leave king in check
- **Missing Assets**: Verify all files in `assets/` directory
- **Missing DLLs (Windows)**: Copy all .dll files to executable directory

# Chapter 11

# Sample Runs

## 11.1 Game User Interface

This section demonstrates the visual interface and functional features of the application during active gameplay.

### 11.1.1 Changing Board Theme

The user can choose a board out of tweleve available colors using the top arrows


Figure 11.1: Board Themes


Figure 11.2: Board Themes


Figure 11.3: Board Themes

### 11.1.2 Main Menu and Navigation

The main menu provides a clean entry point for the user, allowing for quick access to new games or previously saved states.


Figure 11.4: Board Themes

### 11.1.3 Move Visualizations

To assist the player, the UI provides real-time feedback. When a piece is selected, valid moves are highlighted. Additionally, the last move made is visually marked to maintain game flow awareness.


Figure 11.5: Visualization of valid moves (green) and last move (yellow)


Figure 11.6: Visualization of valid moves (green) and last move (yellow)

## 11.2 Game Logic and Terminations

### 11.2.1 Pawn Promotion

A message Box with 4 buttons handle pawn promotion


Figure 11.7: Pawn Promotion

### 11.2.2 Check and Checkmate

The engine constantly monitors the King's safety. When a King is placed in check, the square is highlighted in red. If no legal moves remain, a Checkmate dialog is triggered.

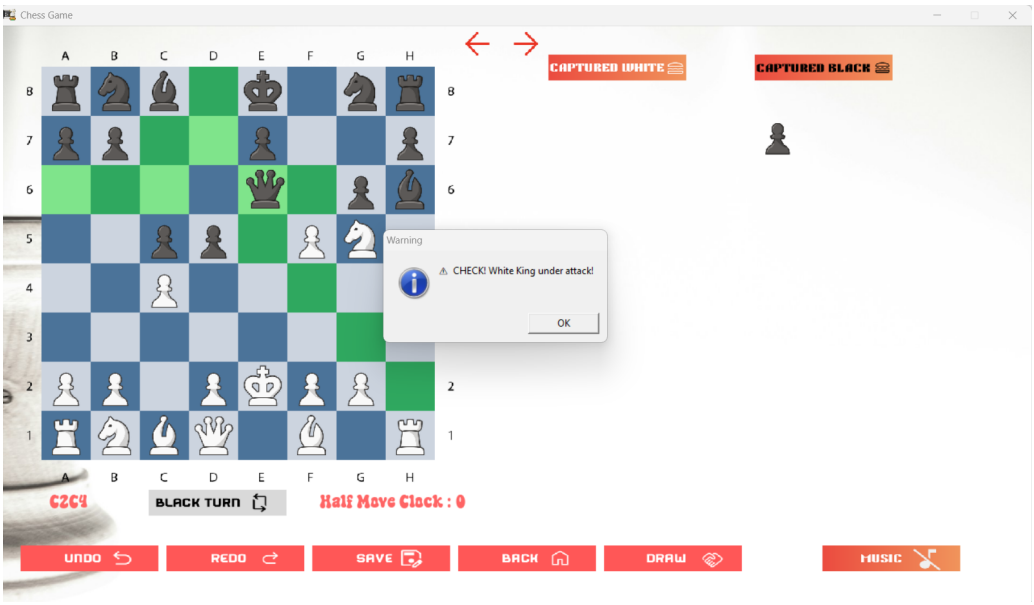


Figure 11.8: Check condition with a warning message


Figure 11.9: Check condition with a warning message

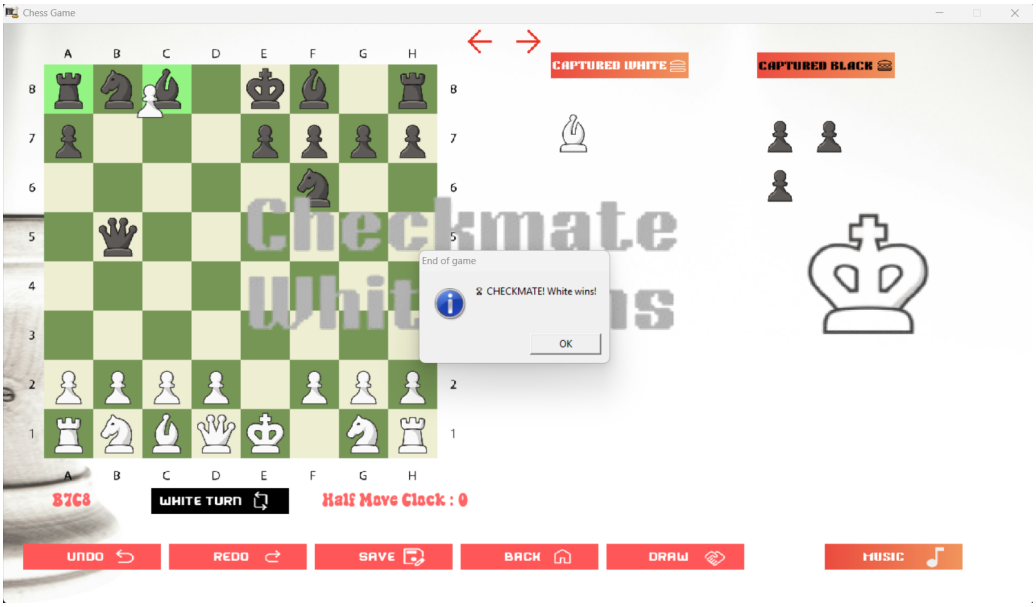


Figure 11.10: Checkmate condition with visual King warning and victory message

### 11.2.3 Save and Load System

The system utilizes eight dedicated slots. Users can access the save menu mid-game to persist their current board state and move history to a binary file.


Figure 11.11: The save slot selection interface

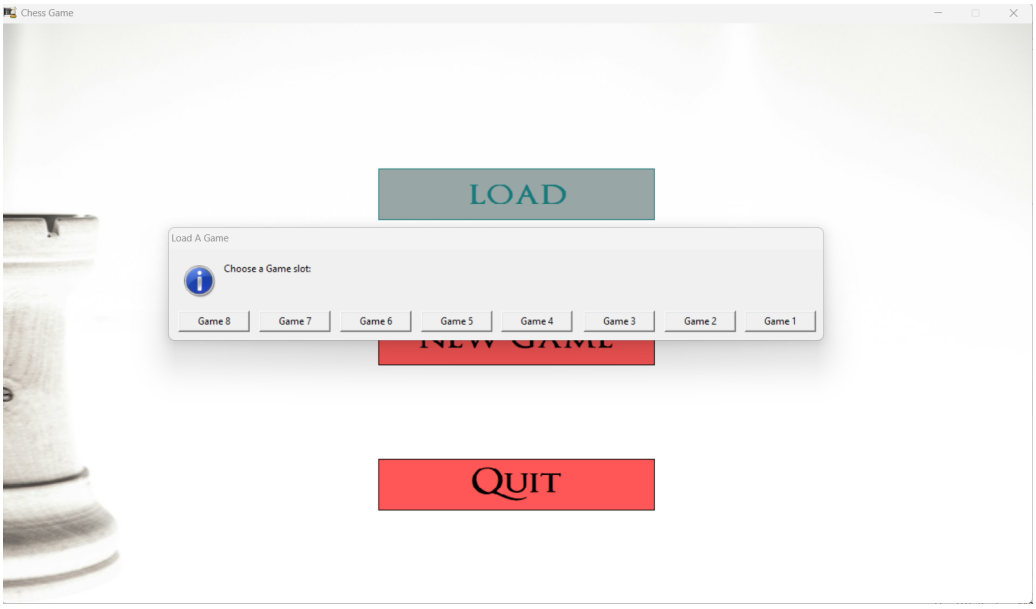


Figure 11.12: The load slot selection interface

### 11.2.4 Draw Conditions

The engine accurately detects draws, including Stalemate and Insufficient Material, Threefold reptition as per FIDE regulations.


Figure 11.13: Threefold detection showing the resulting draw message

# Chapter 12

# References

## 12.1 Books

- K.N. King, *C Programming: A Modern Approach, Second Edition*
  - Chapter 10: Program Organization
  - Chapter 12: Pointers and Arrays
  - Chapter 18: Declarations
  - Chapter 22: Input/Output (Block I/O)

## 12.2 Online Resources

### 12.2.1 Makefile

- https://makefiletutorial.com/
- Stack Overflow: SDL2 CMake undefined reference solutions DSDL MAIN HANDLED flag https://stackoverflow.com/questions/60948791/sdl2-cmake-undefined-refer

### 12.2.2 SDL2 Documentation

- SDL Wiki: https://wiki.libsdl.org/SDL2/FrontPage
- Lazy Foo's SDL Tutorials: https://lazyfoo.net/tutorials/SDL/
- Game Dev Geek SDL2 Tutorials http://gamedevgeek.com/tutorials/getting-started-with-
- Study Plan SDL2 Setup Guide https://www.studyplan.dev/sdl2/sdl-setup-windows#setting-up-sdl2-in-windows-visual-studio

### 12.2.3 Data Structures

- GeeksforGeeks: Stack Data Structure https://www.geeksforgeeks.org/dsa/stack-data-stru

### 12.2.4 Zobrist Hashing

- Chess Programming Wiki: https://www.chessprogramming.org/Zobrist_Hashing
- Wikipedia: Zobrist Hashing https://en.wikipedia.org/wiki/Zobrist_hashing

- Dev.to: Zobrist Hashing Tutorial https://dev.to/larswaechter/zobrist-hashing-72n
- Stack Overflow: Random 64-bit integer generation https://stackoverflow.com/questions/33010010/how-to-generate-random-64-bit-unsigned-integer-in-c

### 12.2.5 C Standard Library

- CPlusPlus.com: `<ctype.h>` https://cplusplus.com/reference/cctype/

### 12.2.6 Miscellaneous

- Stack Overflow: MinGW executable icon compilation https://stackoverflow.com/questions/708238/how-do-i-add-an-icon-to-a-mingw-gcc-compiled-executable

# Chapter 13

# Conclusion

The SDL2 Chess Engine project successfully demonstrates the application of fundamental programming concepts in C to create a fully functional, feature-rich chess game.

- **Data Structure Design**: Efficient use of enumerations, structures, and linked lists for reproducability
- **Algorithm Implementation**: Complex move validation and game state evaluation
- **Memory Management**: Proper allocation and deallocation throughout the application and debugging along the development using valgirand and dr.memory.
- **File I/O**: Binary format saving and loading system
- **Graphical Programming**: SDL2-based user interface with multimedia support
- **Software Engineering**: Modular design with clean separation of concerns

## 13.1 Key Achievements

1. Complete implementation of FIDE chess rules
2. Move validation system preventing illegal moves
3. Efficient game state management with undo/redo functionality
4. Cross-platform compatibility (Windows and Linux)
5. Professional graphical interface with sound effects and board different themes
6. Multiple save slots for game persistence
7. Advanced features like Zobrist hashing for threefold repetition detection using complex bitwise operations

The Chess Engine project serves as a comprehensive demonstration of programming and software development skills acquired through the Computer Programming I course. All credit goes to Prof . Marwan and Eng . Karim for their efforts during this course.